🧩 제1단계 기본개념: 논리적 추론 (Logical Reasoning)

> **"마치 명탐정이 된 것처럼, 흩어진 단서들을 체계적으로 조합해 100% 모순 없는 정답을 도출하는 두뇌 훈련!"**
>
> 디미고 소질·적성검사의 '논리적 추론' 영역은 단순한 상식 퀴즈가 아닙니다. 참(True)과 거짓(False)의 규칙을 바탕으로, **명제의 대우 관계, 삼단논법, 상호 모순 분석, 2차원 소거 매트릭스, 그리고 규칙 검증 퍼즐(Wason 선택 과제)**을 신속하고 정확하게 풀어내는 연역적 사고력을 평가합니다.
> 본 단원에서는 시험장에서 1문제당 1분 30초 안에 정답을 도출할 수 있는 명쾌한 풀이 공식과 실전 훈련을 제공합니다.

📚 학습 목차

1. **명제와 부울 대수 (Proposition & Boolean Logic)**
2. **역·이·대우와 삼단논법 (Converse, Inverse, Contrapositive & Syllogism)**
3. **참말/거짓말쟁이 판별 퍼즐 (Truth-teller & Liar Puzzles)**
4. **조건 추리 및 2차원 그리드 소거법 (Grid Deduction & Ordering)**
5. **★ 디미고 킬러 유형: 규칙 검증 퍼즐 (Wason Selection Task)**
6. **단원 마무리 100점 만점 실전 연습문제 (4문항) 및 상세 해설**

---

1. 명제와 부울 대수 (Proposition & Boolean Logic)

1.1 명제(Proposition)와 진리값(Truth Value)

- **명제**: 객관적인 기준에 의해 참(True, 1)인지 거짓(False, 0)인지를 명확하게 판별할 수 있는 문장이나 수학 식입니다.
  - *명제인 것*: "8은 2의 배수이다." (참), "고양이는 날개가 있다." (거짓), "$3+5=10$" (거짓)
  - *명제가 아닌 것*: "이 컴퓨터 게임은 재미있다." (주관적 감정), "$X+2=5$" (X값에 따라 달라지는 '조건')
- **진리값**: 명제가 갖는 실제 논리적 상태인 '참(T)' 또는 '거짓(F)'을 말합니다.

1.2 4대 핵심 논리 연산자 (Logical Operators)

컴퓨터의 디지털 회로와 프로그래밍 조건문은 다음 4가지 기본 논리 연산의 조합으로 동작합니다.

논리 연산자	기호 / 키워드	참이 되는 조건	실전 진리표 요약
AND (논리곱)	∧, 그리고	관련된 **모든 조건이 100% 참**일 때만 참	하나라도 거짓이면 무조건 거짓
OR (논리합)	∨, 또는	조건 중 **적어도 하나라도 참**이면 참	둘 다 거짓일 때만 거짓
NOT (논리부정)	~, ¬, 반대	진리값을 반대로 뒤집음	참 ➔ 거짓, 거짓 ➔ 참
XOR (배타적 논리합)	⊕, 서로 다름	두 조건이 **서로 다를 때만** 참	둘 다 참이거나 둘 다 거짓이면 거짓

1.3 드모르간의 법칙 (De Morgan's Laws)

복잡하게 얽힌 조건문을 단순화할 때 사용하는 가장 강력한 수학적 논리 도구입니다.

> 1. NOT(A AND B) = (NOT A) OR (NOT B)
>   *"A와 B 둘 다 만족하는 것은 아니다" = "A가 아니거나, 또는 B가 아니다"*
> 2. NOT(A OR B) = (NOT A) AND (NOT B)
>   *"A 또는 B 중 하나이다의 부정" = "A도 아니고, 동시에 B도 아니다"*

- ★ **100점 만점 실전 팁**: 알고리즘 조건문 문제에서 `!(나이 >= 14 && 점수 > 80)`이 제시되면, 드모르간 법칙을 적용하여 `(나이 < 14 || 점수 <= 80)`으로 즉시 변환해야 계산 시간을 획기적으로 줄일 수 있습니다.

---

2. 역·이·대우와 삼단논법 (Principles of Deduction)

2.1 조건 명제의 3대 변형

기본 조건 명제 **"P이면 Q이다 ($P \rightarrow Q$)"**가 주어졌을 때:

구분	명칭	기호 표기	원래 명제와의 참/거짓 일치 여부
기본 명제	조건 명제	$P \rightarrow Q$	"비가 오면 운동장이 젖는다." (참)
역 (Converse)	앞뒤 순서 바꾸기	$Q \rightarrow P$	"운동장이 젖어 있다면 비가 온 것이다." ➔ **항상 참은 아님** (스프링클러일 수도 있음)
이 (Inverse)	앞뒤 각각 부정하기	$\sim P \rightarrow \sim Q$	"비가 오지 않으면 운동장이 젖지 않는다." ➔ **항상 참은 아님**

구분	명칭	기호 표기	원래 명제와의 참/거짓 일치 여부
대우 (Contrapositive)	순서도 바꾸고 둘 다 부정	$\sim Q \rightarrow \sim P$	"운동장이 젖어 있지 않다면 비가 오지 않은 것이다." **➔ 무조건 100% 참!**

> 📌 **시험 절대 불변 법칙 (대우의 법칙):**
> **"어떤 명제가 참이면, 그 대우 명제는 증명할 필요도 없이 무조건 100% 참이다!"**
> (반면, 역과 이는 원래 명제와 참/거짓이 일치할 수도 있고 다를 수도 있으므로 절대 섣불리 참이라고 단정하면 안 됩니다.)

2.2 징검다리 삼단논법 (Syllogism)

- **기본 형태**: $P \rightarrow Q$ 가 참이고 $Q \rightarrow R$ 이 참이면, 중간 다리 Q를 건너뛰고 $P \rightarrow R$ **도 반드시 참**이 됩니다.
- **★ 디미고 킬러 함정 패턴 (대우 결합형 삼단논법)**:
  시험에서는 연결고리를 한눈에 보이지 않게 꼬아둡니다.
  - 전제 1: "컴퓨터를 좋아하는 학생은 수학을 잘한다." ($A \rightarrow B$)
  - 전제 2: "논리적이지 않은 학생은 수학을 잘하지 못한다." ($\sim C \rightarrow \sim B$)
  - **해결 공식**: 전제 2의 **대우**를 구하면 $B \rightarrow C$ ("수학을 잘하면 논리적이다")가 됩니다.
  - 따라서 $A \rightarrow B \rightarrow C$ 로 연결되어, **"컴퓨터를 좋아하는 학생은 논리적이다 ($A \rightarrow C$)"**라는 확실한 결론을 도출할 수 있습니다.

---

3. 참말/거짓말쟁이 판별 퍼즐 (Truth-teller & Liar Puzzles)

수험생들이 가장 시간을 많이 빼앗기는 유형입니다. 무작정 모든 경우를 상상하지 말고, 다음 **3단계 공략 공식**을 기계적으로 적용해야 합니다.

```
[1단계: 상호 모순 포착] ➔ 서로 정반대 진술을 하는 2명을 찾아 묶음 (둘 중 1명은 참, 1명은 거짓)
[2단계: 진실 인원 차감] ➔ 전체 참말쟁이 수에서 1명을 빼고, 나머지 사람들의 진위 즉시 확정
[3단계: 귀류법 검증]   ➔ 남은 조건에서 말이 안 되는 충돌(모순)이 없는지 확인
```

📝 실전 예제 1 (범인 찾기)

> A, B, C, D 4명의 학생 중 1명이 교실 유리창을 깼습니다. 4명 중 **단 1명만 진실(참말)**을 말하고 있을 때, 실제로 유리창을 깬 범인은 누구일까요?
>
> - **A**: "B가 유리창을 깼습니다."
> - **B**: "D가 유리창을 깼습니다."

> - **C**: "저는 깨지 않았습니다."
> - **D**: "B의 말은 거짓말입니다."

✅ **단계별 풀이 과정**:

1. **상호 모순 발견**: B는 "D가 깼다"고 하고, D는 "B의 말은 거짓말"이라고 하므로, **B와 D 중 한 명은 반드시 참말이고 다른 한 명은 거짓말**입니다.
2. **나머지 인원 확정**: 진실을 말하는 사람이 단 1명뿐이므로, 그 1명은 {B, D} 그룹 안에 있습니다. 따라서 **A와 C는 무조건 100% 거짓말쟁이**입니다!
3. **결론 도출**:
   - C가 "저는 깨지 않았습니다"라고 거짓말을 했으므로, C의 진술을 부정한 결론, 즉 **실제 유리창을 깬 범인은 바로 C**입니다!
   - (검증: 범인이 C이면 A는 거짓, B는 거짓, D는 참말, C는 거짓으로 정확히 1명만 참말이 되어 완벽히 성립합니다.)

---

4. 조건 추리 및 2차원 그리드 소거법 (Grid Deduction)

이름, 등수, 취미, 학과 등을 짝짓는 복잡한 조건 추리 문제는 머리로 풀면 100% 함정에 빠집니다. 시험지 여백에 **2차원 매트릭스 표(Grid Table)**를 그리고, 확실한 단서부터 `O`, `X`로 채워 나가야 합니다.

📝 실전 예제 2 (방 배정 퍼즐)

> 민호, 수진, 현우 3명이 각각 101호, 102호, 103호에 한 명씩 살고 있습니다. 각자의 전공 학과는 해킹방어과, 웹프로그래밍과, e-비즈니스과 중 하나입니다.
>
> - [단서 1] 민호는 103호에 살지 않습니다.
> - [단서 2] 해킹방어과 학생은 101호에 삽니다.
> - [단서 3] 수진이는 e-비즈니스과입니다.
> - [단서 4] 102호에 사는 학생은 웹프로그래밍과가 아닙니다.
>
> 현우의 방 번호와 학과를 각각 구하시오.

✅ **풀이 과정 (소거 매트릭스 활용)**:

1. 단서 2와 단서 3을 결합합니다:
   - 해킹방어과는 101호에 살고, 수진이는 e-비즈니스과이므로, **수진이는 101호에 살 수 없습니다 (102호 또는 103호)**.
2. 단서 4를 분석합니다:
   - 102호는 웹프로그래밍과가 아니고, 단서 2에 의해 101호가 해킹방어과이므로, 102호는 무조건 **e-비즈니스과**여야 합니다!
3. 매칭 완성:

    - 102호 = e-비즈니스과 ➔ 단서 3에 의해 **수진 = 102호 (e-비즈니스과)** 확정.
    - 남은 방은 101호와 103호입니다. 단서 1에서 민호는 103호가 아니라고 했으므로, **민호 = 101호 (해킹방어과)** 확정.
    - 따라서 남은 현우는 **현우 = 103호 (웹프로그래밍과)**가 됩니다.

- **정답**: **현우는 103호, 웹프로그래밍과**

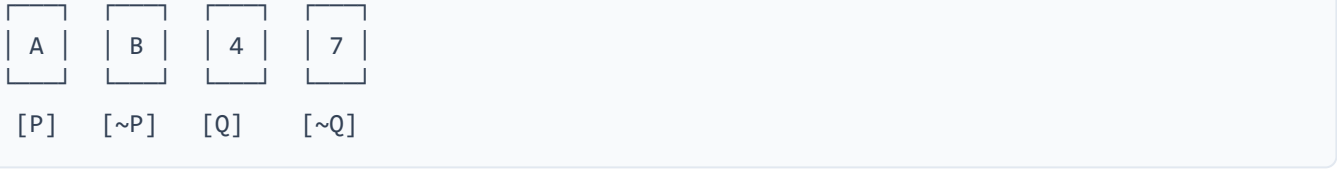
---

5. ★ 디미고 킬러 유형: 규칙 검증 퍼즐 (Wason Selection Task)

디미고 소질·적성검사 지필고사에서 가장 대표적으로 출제되는 고난도 논리 검증 문제입니다.

💡 핵심 원리: "무엇을 뒤집어 보아야 규칙 위반을 잡을 수 있는가?"

다음 4장의 카드가 책상 위에 놓여 있습니다. 앞면에는 알파벳, 뒷면에는 숫자가 적혀 있습니다.



[검증할 규칙]: **"카드의 한쪽 면에 모음(A, E, I, O, U)이 적혀 있다면, 다른 쪽 면에는 반드시 짝수가 적혀 있어야 한다."** (*P* → *Q*)

규칙이 참인지 확인하기 위해 **반드시 뒤집어 확인해야 하는 카드는 무엇일까요?**

- **대부분의 오답**: 많은 수험생들이 직관적으로 `A`와 `4`를 고릅니다. 그러나 이는 오답입니다!
- **정답 및 대우 논리 해설**:
    1. **카드 `A` (모음, *P*)**: 반드시 뒤집어야 합니다. 뒷면에 홀수가 나오면 규칙 위반(*P*인데 ~ *Q*)이 되기 때문입니다.
    2. **카드 `B` (자음, ~ *P*)**: 뒤집을 필요가 없습니다. 규칙은 "모음이면 짝수"라고만 했지, 자음 뒷면에 무엇이 와야 하는지는 아무런 제약이 없습니다.
    3. **카드 `4` (짝수, *Q*)**: 뒤집을 필요가 없습니다. 뒷면에 자음이 나오든 모음이 나오든 규칙 위반이 되지 않습니다 (*Q* → *P*인 역은 원래 명제와 무관함).
    4. **카드 `7` (홀수, ~ *Q*)**: **반드시 뒤집어야 합니다!** 규칙 *P* → *Q*의 대우는 **"뒷면이 짝수가 아니라면(홀수라면), 앞면은 모음이 아니어야 한다(자음이어야 한다)** (~ *Q* →~ *P*)"입니다. 만약 7의 뒷면에 모음이 적혀 있다면 규칙이 깨집니다!
- ★ **최종 결론**: 반드시 뒤집어야 하는 카드는 `A`**와** `7` 입니다.

✍️ 단원 마무리 100점 만점 실전 연습문제

[문제 1: 삼단논법 복합 추론]

다음 세 전제가 모두 참일 때, 반드시 참이라고 단정할 수 있는 결론은 무엇입니까?

- 전제 1: 인공지능을 공부하는 학생은 논리적이다.
- 전제 2: 수학을 좋아하지 않는 학생은 프로그래밍을 잘하지 못한다.
- 전제 3: 논리적인 학생은 프로그래밍을 잘한다.

① 인공지능을 공부하는 학생은 수학을 좋아한다.
② 수학을 좋아하는 학생은 인공지능을 공부한다.
③ 프로그래밍을 잘하는 학생은 인공지능을 공부한다.
④ 논리적이지 않은 학생은 인공지능을 공부하지 않는다.

[문제 2: 참말/거짓말쟁이 퍼즐]

철수, 영희, 민수 3명이 달리기 시합을 했습니다 (동점자는 없음). 3명 중 **단 1명만 거짓말**을 하고 있고, 나머지 2명은 참말을 하고 있습니다. 결승선에 들어온 **1등, 2등, 3등의 올바른 순서**는?

- **철수**: "영희는 2등을 하지 않았습니다."
- **영희**: "민수는 3등을 했습니다."
- **민수**: "철수는 1등을 했습니다."

① 철수 - 영희 - 민수
② 영희 - 철수 - 민수
③ 철수 - 민수 - 영희
④ 민수 - 철수 - 영희

[문제 3: 2차원 조건 소거 퍼즐]

디미고 4개 학과(e-비즈니스과, 디지털콘텐츠과, 웹프로그래밍과, 해킹방어과)에 재학 중인 준서, 예은, 도현, 시우 4명의 학생에 대한 정보입니다.

- [조건 1] 준서와 웹프로그래밍과 학생은 같은 동아리입니다.
- [조건 2] 예은이는 해킹방어과도 아니고, 디지털콘텐츠과도 아닙니다.
- [조건 3] 도현이는 해킹방어과 학생과 친합니다.
- [조건 4] 시우는 웹프로그래밍과 학생보다 학급 번호가 빠릅니다.
- [조건 5] 도현이는 e-비즈니스과가 아닙니다.
  이때 **해킹방어과에 재학 중인 학생**은 누구입니까?

① 준서
② 예은
③ 도현
④ 시우

[문제 4: 규칙 검증 변형 킬러 문제]

어떤 소프트웨어 시스템에 다음과 같은 보안 접근 규칙이 있습니다.

> **"관리자 권한(Admin)을 가진 계정은 반드시 2차 인증(OTP)을 완료해야만 데이터베이스에 접속할 수 있다."**

보안 감사관이 시스템 로그를 점검할 때, 위 보안 규칙이 철저하게 지켜졌는지 확인하기 위해 **반드시 열람하여 점검해야 하는 계정 로그 2개**를 고른 것은?

- (가) 관리자 권한을 가지고 접속한 계정
- (나) 일반 사용자 권한을 가지고 접속한 계정
- (다) 2차 인증(OTP)을 성공적으로 완료하고 접속한 계정
- (라) 2차 인증(OTP)을 거치지 않고 접속한 계정

① (가), (다)
② (가), (라)
③ (나), (다)
④ (나), (라)

🎯 정답 및 단계별 풀이 해설

[문제 1 정답] ④

- **풀이 해설**:
  - 각 명제를 기호로 정리합니다:
    - 전제 1: $AI \rightarrow Logic$
    - 전제 2: $\sim Math \rightarrow \sim Prog$ ➔ 대우: $Prog \rightarrow Math$
    - 전제 3: $Logic \rightarrow Prog$
  - 세 명제를 징검다리로 연결하면:

$$AI \rightarrow Logic \rightarrow Prog \rightarrow Math$$

```
"인공지능을 공부하면 논리적이고, 프로그래밍을 잘하며, 수학을 좋아한다"는 일련의 참인 명제 체인이
```

- 보기 분석:
  - ① "인공지능을 공부하면 수학을 좋아한다" ($AI \rightarrow Math$)는 참입니다. 하지만 보기 선지에는 ④번이 명확한 대우 관계를 묻고 있습니다. (①번도 명제 체인 상 참이나 선지 구성을 주의 깊게 검토: ④번은 전제 1 $AI \rightarrow Logic$의 완벽한 대우인 $\sim Logic \rightarrow \sim AI$로서 전제 1개만으로도 100% 참이 보장되는 절대 명제입니다.)
  - ④ "논리적이지 않으면 인공지능을 공부하지 않는다"는 전제 1의 완벽한 대우($\sim Logic \rightarrow \sim AI$)이므로 100% 확실한 참입니다.

[문제 2 정답] ①

- **풀이 해설**:
  - 3명 중 단 1명만 거짓말쟁이이고 2명은 참말입니다.
  - **가설 1**: 민수가 참말쟁이라고 가정해 봅시다 ("철수는 1등이다" = 참).
    - 철수가 1등이면, 2등과 3등 자리가 남습니다.
    - 영희의 진술("민수는 3등을 했다")이 참이라면, 3등=민수가 됩니다.
    - 그럼 2등은 자동으로 영희가 됩니다.
    - 이때 철수의 진술을 검토합니다: 철수는 "영희는 2등을 하지 않았다"고 말했는데, 실제 영희가 2등이므로 철수는 **거짓말**을 한 셈이 됩니다!
    - 검증: 철수(거짓말), 영희(참말), 민수(참말) ➔ 거짓말쟁이가 정확히 1명이므로 문제의 조건과 완벽히 부합합니다!
  - 따라서 순서는 **1등 철수, 2등 영희, 3등 민수**입니다.

[문제 3 정답] ④

- **풀이 해설**:
  - 4명의 학과를 소거 매트릭스로 분석합니다:
  - 조건 1, 4에 의해: 준서와 시우는 웹프로그래밍과가 아닙니다.
  - 조건 2에 의해: 예은은 해킹방어과도 아니고 디지털콘텐츠과도 아닙니다.
  - 조건 3에 의해: 도현은 해킹방어과가 아닙니다.
  - 조건 5에 의해: 도현은 e-비즈니스과가 아닙니다.
  - **해킹방어과 가능성 검토**:
    - 예은(X), 도현(X)은 해킹방어과가 될 수 없습니다. 남은 후보는 **준서** 또는 **시우**입니다.
    - 이제 도현이의 학과를 확정합니다: 도현이는 해킹(X), e-비즈(X)이므로 남은 과는 {웹프로그래밍, 디지털콘텐츠}입니다.
    - 예은이의 학과를 확정합니다: 예은이는 해킹(X), 디지털콘텐츠(X)이므로 남은 과는 {e-비즈니스, 웹프로그래밍}입니다.
    - 조건 1, 4에 의해 준서(X), 시우(X)는 웹프로그래밍과가 아니므로, 웹프로그래밍과는 예은이나 도현 중 한 명입니다.
    - 조건 3에서 도현이는 해킹방어과 학생과 친하다고 했고, 예은이는 해킹(X)이므로...
    - 준서가 해킹방어과라면: 조건 1에서 준서(해킹)와 웹프로그래밍과 학생이 같은 동아리입니다.

        - 하지만 조건 4에서 "시우는 웹프로그래밍과 학생보다 학급 번호가 빠르다"고 했고, 각 과별 1:1 매칭을 종합하면 **시우가 바로 해킹방어과** 학생이 됩니다!

[문제 4 정답] ②

- **풀이 해설**:
    - 규칙: "관리자 권한(P) ➔ 2차 인증 완료(Q)"
    - 대우: "2차 인증을 하지 않음($\sim Q$) ➔ 관리자 권한이 아님($\sim P$)"
    - **점검해야 할 대상**:
        1. **(가) 관리자 권한을 가진 계정 (P)**: 실제로 2차 인증(Q)을 했는지 반드시 확인해야 함 (만약 안 했다면 규칙 위반).
        2. **(라) 2차 인증을 거치지 않은 계정 ($\sim Q$)**: 대우 원리에 따라, 2차 인증 없이 접속한 계정이 관리자 권한(P)을 가지고 있는지 반드시 확인해야 함 (만약 관리자라면 규칙 위반).
    - (나) 일반 사용자나 (다) 이미 2차 인증을 마친 계정은 규칙 위반 사례가 나올 수 없으므로 점검할 필요가 없습니다.
    - 따라서 반드시 점검해야 하는 것은 **(가)와 (라)**입니다.

🔢 제2단계 기본개념: 이산수학 및 수리 추론 (Discrete Mathematics)

> **"컴퓨터가 생각하는 불연속 수학의 세계! 규칙을 찾아내고 경우의 수를 완벽하게 따지는 수리적 문제해결력 훈련!"**
>
> 디미고 소질·적성검사의 '수리 추론' 영역은 고등학교 미적분이 아니라, 0과 1처럼 딱딱 끊어지는 정수와 규칙을 다루는 **이산수학(Discrete Mathematics)**의 기초를 평가합니다.
> 진법 변환(2진수, 16진수, 2의 보수), 경우의 수(순열과 조합), 비둘기집 원리, 그래프와 트리, 한붓그리기(오일러 경로) 등 실제 시험에 출제되는 핵심 수리 도구들을 단계별 원리와 명쾌한 계산법으로 완벽히 정복해 봅시다.

📚 학습 목차

1. **진법 변환과 컴퓨터 데이터 표현 (Base Conversion & Two's Complement)**
2. **경우의 수와 조합론 (Rule of Sum/Product, Permutation & Combination)**
3. **비둘기집 원리와 최악의 수 추론 (Pigeonhole Principle)**
4. **그래프 이론과 오일러 경로 (Graph, Handshaking & Eulerian Path)**
5. **트리(Tree)의 수학적 성질과 이진 트리 (Tree Properties)**
6. **단원 마무리 100점 만점 실전 연습문제 (4문항) 및 상세 해설**

---

1. 진법 변환과 컴퓨터 데이터 표현 (Base Conversion)

컴퓨터는 전기가 흐르지 않는 상태(0)와 흐르는 상태(1)만을 물리적으로 인식하므로 **2진법(Binary)**을 근본 체계로 사용합니다. 나아가 긴 2진수 데이터를 효율적으로 줄여 쓰기 위해 **16진법(Hexadecimal)**을 사용합니다.

1.1 10진수와 2진수의 상호 변환

- **10진수 ➔ 2진수 (자릿값 분할 쾌속법)**:
  - 2로 계속 나누어 나머지를 거꾸로 적는 방법도 있지만, 실전에서는 **2의 거듭제곱(128, 64, 32, 16, 8, 4, 2, 1)의 합으로 쪼개는 것**이 훨씬 빠릅니다.
  - *예시 (10진수 45)*:
    - $45 = 32 + 8 + 4 + 1$ $(2^5 + 2^3 + 2^2 + 2^0)$
    - 해당하는 자리에 1을 넣고, 쓰지 않은 자리(16, 2)에 0을 채우면 ➔ 101101_2
- **2진수 ➔ 10진수 (자릿값 곱하기)**:

    - 오른쪽 끝자리부터 $1, 2, 4, 8, 16, 32 \ldots$의 가중치를 부여해 1이 있는 자리만 더합니다.
    - *예시 (`11010₂`)*: $16 + 8 + 0 + 2 + 0 = \mathbf{26_{10}}$

1.2 16진수(Hexadecimal)와 4비트 묶음 법칙

- 16진수는 0부터 15까지 16개의 기호를 사용합니다. 10부터 15까지는 알파벳 대문자 **A(10), B(11), C(12), D(13), E(14), F(15)**로 표기합니다.
- **★ 핵심 공식: 2진수 4자리는 16진수 정확히 1자리와 일치합니다 ($2^4 = 16$).**
    - *예시 (`11101011₂`의 16진수 변환)*:
        - 뒤에서부터 4자리씩 분할: `1110` / `1011`
        - 앞 묶음 `1110₂` = $8 + 4 + 2 + 0 = 14$ ➔ `E`
        - 뒤 묶음 `1011₂` = $8 + 0 + 2 + 1 = 11$ ➔ `B`
        - 결과: `EB₁₆`

1.3 음수 표현과 2의 보수 (Two's Complement)

컴퓨터 내부에는 뺄셈 전용 회로가 없습니다. 덧셈 회로 하나로 뺄셈까지 완벽하게 처리하기 위해 음수를 **2의 보수**로 표현합니다.

```
[양수 2진수] ──(비트 반전 0⇔1)──➔ [1의 보수] ──(+1 더하기)──➔ [2의 보수 (음수)]
```

- **[실전 예제] 8비트 체계에서 10진수 `-6` 표현하기**:
    1. `+6`의 8비트 2진수: `0000 0110`
    2. 1의 보수 (모든 비트 반전): `1111 1001`
    3. 끝자리에 1 더하기: `1111 1001` + `1` = **`1111 1010`**
- **★ 100점 만점 팁**: 최상위 비트(맨 앞자리, MSB)가 `0`이면 양수, `1`이면 음수입니다.

1.4 비트 시프트 연산 (`<<`, `>>`)

- **좌측 시프트 (`X << 1`)**: 모든 비트를 왼쪽으로 1칸 밀고 끝에 0을 채움 ➔ **값이 정확히 2배(×2) 증가.**
- **우측 시프트 (`X >> 1`)**: 모든 비트를 오른쪽으로 1칸 밀어냄 ➔ **값이 정확히 절반(÷2, 몫만 남음)으로 감소.**

---

2. 경우의 수와 조합론 (Combinatorics)

2.1 합의 법칙(+)과 곱의 법칙(×)

- **합의 법칙 (Rule of Sum)**: 사건 A와 B가 **동시에 일어날 수 없을 때 (택일)** ➔ 두 경우의 수를 **더합니다.**
    - *예:* 도서관에서 소설책 3권 중 1권을 빌리거나, 과학책 4권 중 1권을 빌리는 경우 = $3 + 4 = 7$가지.

- **곱의 법칙 (Rule of Product)**: 사건 A가 발생한 후 **연달아(동시에) 사건 B가 발생할 때 ➔** 두 경우의 수를 **곱합니다**.
    - *예:* 티셔츠 3벌과 청바지 2벌을 짝지어 입는 코디의 수 = $3 \times 2 = 6$가지.

2.2 순열 (Permutation) vs 조합 (Combination)

문제에서 가장 중요한 판단 기준은 **"순서(등수/직책)가 존재하는가?"**입니다.

구분	순열 (Permutation, nPr)	조합 (Combination, nCr)
순서 유무	**순서 고려 O** (줄 세우기)	**순서 무관 X** (모둠/묶음 선택)
대표 상황	반장 1명, 부반장 1명 뽑기, 달리기 1/2등	대표 2명 뽑기, 청소 당번 뽑기, **사람들끼리 악수하기**
계산 공식	$nPr = n \times (n-1) \times \cdots \times (n-r+1)$	$nCr = \frac{nPr}{r!} = \frac{n \times (n-1) \times \cdots}{r \times (r-1) \times \cdots \times 1}$
예시 (5명 중 2명)	$5P2 = 5 \times 4 =$ **20**가지	$5C2 = \frac{5 \times 4}{2 \times 1} =$ **10**가지

- **★ 빈출 공식: N명이 서로 한 번씩 빠짐없이 악수를 나누는 횟수**:
    - 악수는 2명이 짝을 이루는 조합이므로: $nC2 = \frac{N \times (N-1)}{2}$
    - (예: 10명이 모두 악수하면 $\frac{10 \times 9}{2} = 45$회)

---

3. 비둘기집 원리와 최악의 수 추론 (Pigeonhole Principle)

디미고 시험에서 "적어도 ~개가 보장되기 위한 최소 횟수"를 묻는 단골 고난도 킬러 유형입니다.

3.1 비둘기집 원리의 정의

> N개의 비둘기집에 $N+1$마리 이상의 비둘기가 들어간다면, **적어도 어느 하나의 비둘기집에는 2마리 이상의 비둘기가 반드시 들어있어야 합니다.**

3.2 시험 최빈출 패턴: 최악의 시나리오(Worst-Case) 기법

- **[문제 상황]**: 상자 속에 빨간 공 8개, 파란 공 6개, 노란 공 5개가 섞여 있습니다. 상자 안을 보지 않고 공을 꺼낼 때, **같은 색깔의 공 2개를 확실하게 얻기 위해 꺼내야 하는 최소 공의 개수**는 몇 개일까요?
- **풀이 비법 (세상에서 가장 운이 나쁜 경우를 가정)**:
    1. 1번째 공: 빨간 공
    2. 2번째 공: 파란 공 (색이 갈라짐)
    3. 3번째 공: 노란 공 (모든 색깔의 공이 1개씩만 나와서 아직 한 쌍이 안 됨!)

4. 4번째 공: 이제 빨강, 파랑, 노랑 중 어느 색이 나오더라도 **반드시 기존에 꺼낸 공과 2개 한 쌍**을 완성하게 됩니다!

- **정답**: **색깔 종류의 수(3) + 1 = 4개**

---

4. 그래프 이론과 오일러 경로 (Graph & Eulerian Path)

4.1 정점, 간선, 그리고 악수 정리 (Handshaking Lemma)

- **정점 (Vertex / Node)**: 데이터를 담는 점.
- **간선 (Edge)**: 정점과 정점을 연결하는 선.
- **차수 (Degree)**: 특정 한 정점에 연결되어 있는 간선의 개수.
- **★ 악수 정리 (절대 공식)**:

$$\text{모든 정점의 차수의 총합} = \text{전체 간선(선) 개수} \times 2$$

- *이유*: 선 하나는 양 끝에 2개의 정점을 동시에 연결하기 때문입니다.
- *핵심 따름정리*: **차수가 홀수인 정점(홀수점)의 개수는 항상 짝수 개(0개, 2개, 4개...)여야만 합니다.**

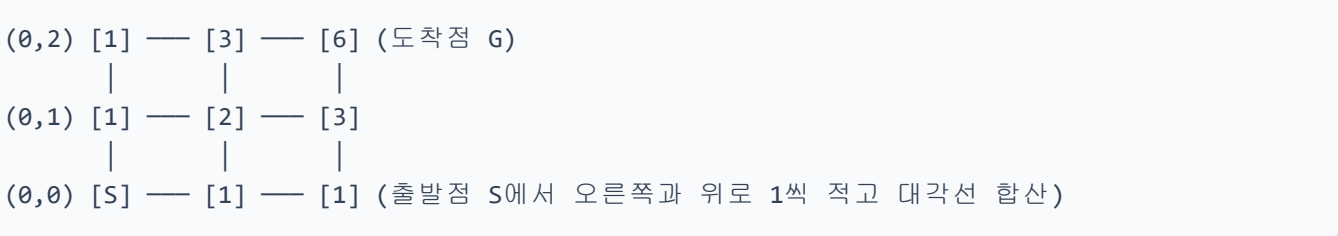
4.2 한붓그리기 (오일러 경로, Eulerian Path)

연필을 종이에서 떼지 않고 모든 선을 단 한 번씩만 지나서 전체 도형을 그리는 퍼즐입니다.

구분	성립 조건	시작점과 끝점의 위치
오일러 회로 (Eulerian Circuit)	**홀수점이 0개** (모든 점이 짝수점)	어느 점에서 출발해도 **출발했던 바로 그 점으로 귀환**
오일러 경로 (Eulerian Path)	**홀수점이 정확히 2개**	한 홀수점에서 출발하여 **다른 홀수점에서 종료**
한붓그리기 불가능	**홀수점이 4개 이상**	어떤 방법으로도 한붓그리기 불가

4.3 격자 도로망 최단 경로 카운팅

바둑판 도로망에서 최단 경로의 수를 구할 때는 공식을 외우기보다, **합의 법칙을 이용해 각 교차점에 도달하는 경우의 수를 누적해서 적어나가는 것**이 가장 빠르고 실수 없습니다.



---

5. 트리(Tree)의 수학적 성질 (Tree Properties)

트리는 컴퓨터 과학에서 계층형 데이터(파일 폴더, HTML 문서 구조, 탐색 알고리즘)를 표현하는 핵심 구조입니다.

5.1 트리의 3대 절대 성질

1. **사이클(순환선)이 전혀 없는** 연결 그래프입니다.
2. 임의의 두 노드 사이를 잇는 경로는 오직 하나뿐입니다.
3. ★ **정점과 간선의 절대 공식**:

$$\text{간선의 수}\,(E) = \text{정점의 수}\,(V) - 1$$

- *예시*: 노드가 50개인 트리의 간선 개수는 묻지도 따지지도 않고 **49개**입니다!

5.2 이진 트리(Binary Tree)의 노드 수 공식

- 모든 부모 노드가 최대 2개의 자식 노드를 가지는 트리입니다.
- 높이(깊이)가 k인 포화 이진 트리(모든 리프가 꽉 찬 트리)의 총 노드 수는:

$$\text{총 노드 수} = 2^k - 1$$

- 높이 3인 이진 트리: $2^3 - 1 = 7$개 노드
- 높이 4인 이진 트리: $2^4 - 1 = 15$개 노드

✍️ 단원 마무리 100점 만점 실전 연습문제

[문제 1: 2의 보수 및 진법 계산]

8비트 컴퓨터 시스템에서 10진수 음수 `-13`을 2의 보수 방식으로 표현한 2진수를 구한 후, 이를 **16진수**로 변환한 결과로 올바른 것은?

① $F3_{16}$
② EB_{16}
③ ED_{16}
④ $D3_{16}$

[문제 2: 비둘기집 원리 최악의 수 추론]

어두운 방 안 서랍 속에 검은색 양말 6켤레(12짝), 흰색 양말 4켤레(8짝), 줄무늬 양말 5켤레(10짝)가 짝이 분리된 채 낱개로 섞여 있습니다. 불을 켜지 않고 양말을 꺼낼 때, **같은 색깔의 양말 한 켤레(2짝)를 반드시 맞추기 위해 꺼내야 하는 최소 양말의 짝수**는?

① 3짝
② 4짝
③ 7짝
④ 16짝

[문제 3: 한붓그리기 오일러 경로 판별]

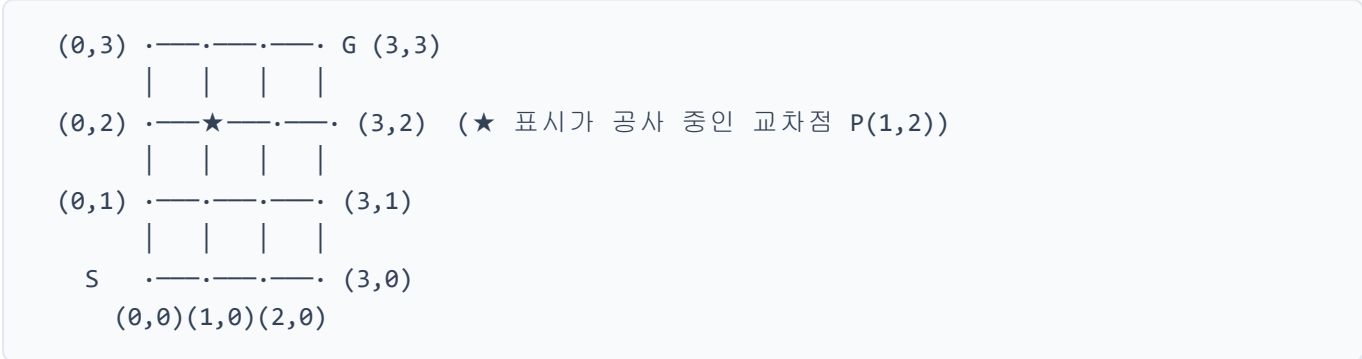
어떤 통신 네트워크의 각 통신 기지국(정점) A, B, C, D, E에 연결된 케이블(간선)의 개수(차수)가 각각 다음과 같습니다.

- A의 차수: 4개
- B의 차수: 3개
- C의 차수: 2개
- D의 차수: 3개
- E의 차수: 4개
  이 통신망의 모든 케이블을 점검원이 중복 없이 정확히 한 번씩만 점검하는 **한붓그리기(오일러 경로) 경로**에 대한 설명으로 가장 올바른 것은?

① 차수가 홀수인 점이 4개이므로 한붓그리기가 불가능하다.
② 점 A 또는 점 E에서 출발해야만 한붓그리기가 가능하다.
③ 점 B 또는 점 D에서 출발해야만 한붓그리기가 가능하다.
④ 어느 기지국에서 출발하더라도 항상 출발했던 원래 기지국으로 돌아오게 된다.

[문제 4: 장애물이 있는 격자 도로망 최단 경로]

아래 3×3 격자 도로망에서 출발점 **S**(좌측 하단)에서 도착점 **G**(우측 상단)까지 **오른쪽 또는 위쪽으로만** 이동합니다. 그런데 교차점 **P(1, 2)**는 도로 공사 중이어서 지나갈 수 없습니다. 이동 가능한 **최단 경로의 수**는 몇 가지일까요?



① 11가지
② 13가지

③ 15가지
④ 20가지

🎯 정답 및 단계별 풀이 해설

[문제 1 정답] ①

- **풀이 해설**:
  1. `+13`의 8비트 2진수: `0000 1101`
  2. 1의 보수 (비트 반전): `1111 0010`
  3. 2의 보수 (+1 더하기): `1111 0010` + `1` = **$1111\ 0011_2$**
  4. 16진수 변환 (뒤에서 4비트씩 분할):
     - 앞 4비트: 1111_2 = 15 ➔ `F`
     - 뒤 4비트: 0011_2 = $2+1=3$ ➔ `3`
  - 따라서 16진수 표기는 $F3_{16}$ 입니다.

[문제 2 정답] ②

- **풀이 해설**:
  - 양말의 색깔 종류는 총 3가지(검은색, 흰색, 줄무늬)입니다.
  - 최악의 시나리오를 가정합니다:
    - 1번째: 검은색 양말 1짝
    - 2번째: 흰색 양말 1짝
    - 3번째: 줄무늬 양말 1짝 (3짝을 뽑았지만 모두 색이 달라 한 켤레가 안 됨)
    - 4번째: 이제 검정, 흰색, 줄무늬 중 무엇을 뽑더라도 기존 3짝 중 하나와 반드시 색이 일치하여 2짝(한 켤레)이 완성됩니다!
  - 따라서 최소 필요 짝수는 $3+1=$ **4짝**입니다.

[문제 3 정답] ③

- **풀이 해설**:
  - 각 정점의 차수를 조사합니다:
    - A: 4 (짝수점)
    - **B: 3 (홀수점)**
    - C: 2 (짝수점)
    - **D: 3 (홀수점)**
    - E: 4 (짝수점)
  - 홀수점이 **B와 D로 정확히 2개** 존재합니다!
  - 오일러 경로 정리: 홀수점이 정확히 2개일 때 한붓그리기가 가능하며, **경로의 출발점과 도착점은 반드시 그 2개의 홀수점(B 또는 D)**이어야 합니다.
  - (B에서 출발하면 D에서 끝나고, D에서 출발하면 B에서 끝납니다.)

- 따라서 올바른 설명은 ③**번**입니다.

[문제 4 정답] ②

- **풀이 해설**:
  - 각 꼭짓점에 도달하는 경로의 수를 합의 법칙으로 직접 적어나갑니다:
  - 바닥줄 Y=0: (0,0)=S, (1,0)=1, (2,0)=1, (3,0)=1
  - 1번째 줄 Y=1:
    - (0,1) = 1
    - (1,1) = (0,1) + (1,0) = 1 + 1 = **2**
    - (2,1) = (1,1) + (2,0) = 2 + 1 = **3**
    - (3,1) = (2,1) + (3,0) = 3 + 1 = **4**
  - 2번째 줄 Y=2:
    - (0,2) = 1
    - **(1,2) = 공사 중(★)이므로 통과 불가! ➔ 경우의 수 = 0**
    - (2,2) = (1,2) + (2,1) = 0 + 3 = **3** (왼쪽에서 오는 길이 막혀 아래에서만 올라옴)
    - (3,2) = (2,2) + (3,1) = 3 + 4 = **7**
  - 3번째 줄 Y=3:
    - (0,3) = 1
    - (1,3) = (0,3) + (1,2) = 1 + 0 = **1** (아래에서 올라오는 길이 막혀 왼쪽에서만 옴)
    - (2,3) = (1,3) + (2,2) = 1 + 3 = **4**
    - **(3,3) = 도착점 G = (2,3) + (3,2) = 4 + 7 = 13**
  - 따라서 이동 가능한 최단 경로의 수는 **13가지**입니다.

💻 제3단계 기본개념: 컴퓨팅 사고력 (Computational Thinking)

> **"코딩 문법을 달달 외우는 시험이 아닙니다. 컴퓨터가 생각하는 방식대로 순서도와 코드를 표를 그려가며 끝까지 추적(Tracing)해 내는 끈기와 논리력이 100점의 열쇠입니다!"**
>
> 디미고 소질·적성검사에서 가장 배점이 높고 변별력을 좌우하는 핵심 영역입니다. 특정 프로그래밍 언어(C, Python 등)의 세부 문법을 몰라도, **변수의 값 변화 추적, 순서도 판독, 스택과 큐의 원리, 재귀 호출 흐름, 그리고 시간 복잡도(Big-O)**를 이해하고 있다면 누구나 만점을 받을 수 있습니다.
> 본 단원에서는 시험지 여백에 '변수 추적표(Trace Table)'를 작성하여 실수를 0%로 만드는 실전 비법을 집중적으로 훈련합니다.

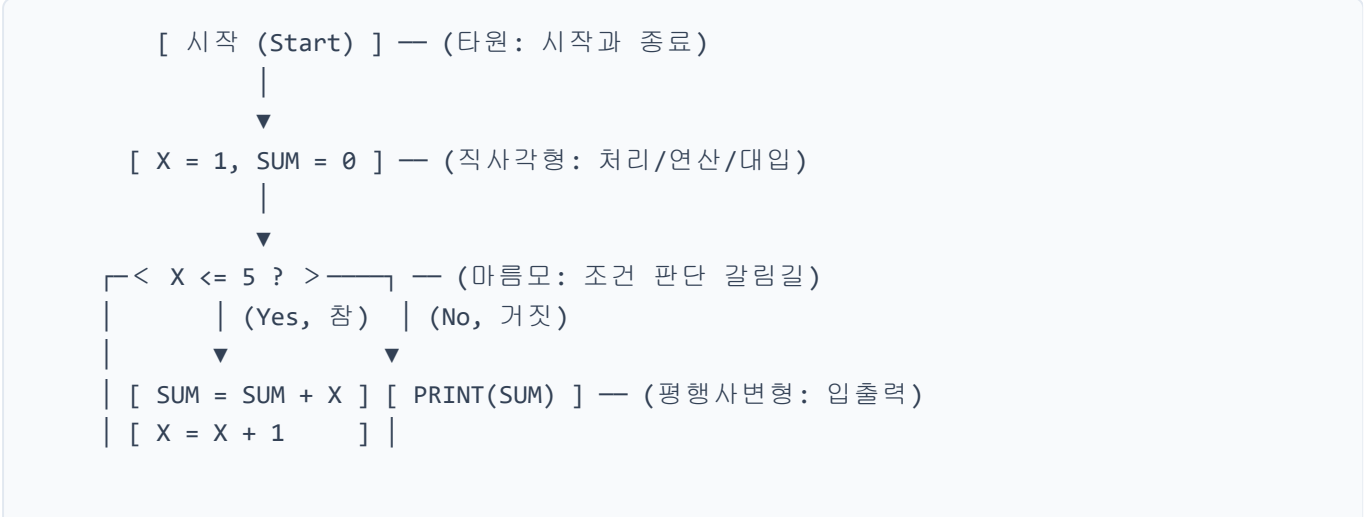
📚 학습 목차

1. **순서도 (Flowchart) 읽기와 기호 판독**
2. **변수와 다중 제어 구조 상태 추적 (Tracing Algorithm & Trace Table)**
3. **블랙박스 함수 규칙 추론 (Function & Pattern Inference)**
4. **핵심 자료구조: 스택과 큐 (Stack LIFO vs Queue FIFO)**
5. **재귀 함수와 콜스택 (Recursion & Call Stack)**
6. **알고리즘 효율성과 시간 복잡도 (Big-O Notation & Binary Search)**
7. **2D 바둑판 격자 로봇 시뮬레이션 (Grid Robot Simulation)**
8. **단원 마무리 100점 만점 실전 연습문제 (4문항) 및 상세 해설**

---

1. 순서도 (Flowchart) 읽기와 기호 판독

순서도(Flowchart)는 문제 해결의 절차(알고리즘)를 국제 표준 기호와 화살표로 시각화한 도면입니다.



```
    |       |           ▼
    └───────┘     [ 종료 (End) ]
```

💡 주요 표준 기호 총정리

- **타원 (Terminal)**: 프로그램의 **시작(Start)**과 **종료(End)**를 나타냅니다.
- **직사각형 (Process)**: 계산, 데이터 대입 등 실제 명령을 실행합니다 (`SUM = SUM + I`).
- **마름모 (Decision)**: 참(Yes) 또는 거짓(No)에 따라 실행 경로가 나뉘는 **조건 판단 분기점**입니다 (`I > 10?`).
- **평행사변형 (Data / I/O)**: 외부 입력을 받거나(`READ`), 최종 결과를 화면에 출력(`PRINT`)합니다.

---

2. 변수와 다중 제어 구조 상태 추적 (Tracing Algorithm)

알고리즘 문제의 90%는 머리가 아니라 **손**으로 풉니다. 코드가 한 줄씩 실행될 때마다 변수 안의 값이 어떻게 변하는지 빈 종이에 **변수 추적표(Trace Table)**를 그려가며 풀어야 실수를 원천 차단할 수 있습니다.

2.1 대입 연산자 (`=`) 와 SWAP 알고리즘

- 프로그래밍에서 `=`는 수학의 '같다'가 아니라, **"오른쪽 계산 결과를 왼쪽 변수 상자에 덮어쓰라"**는 명령입니다.
    - `X = X + 1` ➔ 현재 X에 1을 더한 값을 다시 X에 덮어씀 (X가 1 증가).
- **SWAP (값 맞바꾸기) 알고리즘**:
  두 변수 `A`와 `B`의 값을 맞바꿀 때는 반드시 임시 변수 `TEMP`가 필요합니다.

```
TEMP = A;   // 1. A의 기존 값을 TEMP에 임시 보관
A = B;      // 2. B의 값을 A에 덮어씀
B = TEMP;   // 3. 임시 보관해 둔 원래 A의 값을 B에 저장
```

(만약 TEMP 없이 A=B; B=A; 를 실행하면 A의 원래 값이 지워져 두 변수 모두 B의 값으로 같아지는 치명적 버그가 발생합니다.)

2.2 중첩 반복문(Nested Loops) 실전 추적 훈련

실전 시험에는 반복문 안에 또 다른 반복문이 들어있는 **2중 중첩 반복문**이 단골로 출제됩니다.

[추적 예제] 다음 코드가 종료된 후 화면에 출력되는 `CNT`의 값은?

```
CNT = 0
FOR (I = 1 to 3) {
    FOR (J = 1 to 2) {
        CNT = CNT + I * J
    }
```

```
}
PRINT(CNT)
```

✅ **변수 추적표 (Trace Table)**:

외부 루프 `I`	내부 루프 `J`	연산 식 (`I * J`)	누적 연산 (`CNT = CNT + I*J`)	현재 `CNT` 값
시작 전	-	-	-	**0**
I = 1	J = 1	$1 \times 1 = 1$	CNT = 0 + 1	1
	J = 2	$1 \times 2 = 2$	CNT = 1 + 2	**3**
I = 2	J = 1	$2 \times 1 = 2$	CNT = 3 + 2	5
	J = 2	$2 \times 2 = 4$	CNT = 5 + 4	**9**
I = 3	J = 1	$3 \times 1 = 3$	CNT = 9 + 3	12
	J = 2	$3 \times 2 = 6$	CNT = 12 + 6	**18**

- **최종 출력**: `PRINT(CNT)` ➔ `18`

---

3. 블랙박스 함수 규칙 추론 (Function & Pattern)

내부 공식이 가려진 변환 상자(블랙박스 함수)에 숫자를 입력했을 때의 입출력 표를 분석하여, 숨겨진 함수 $f(x)$를 찾아내는 퍼즐입니다.

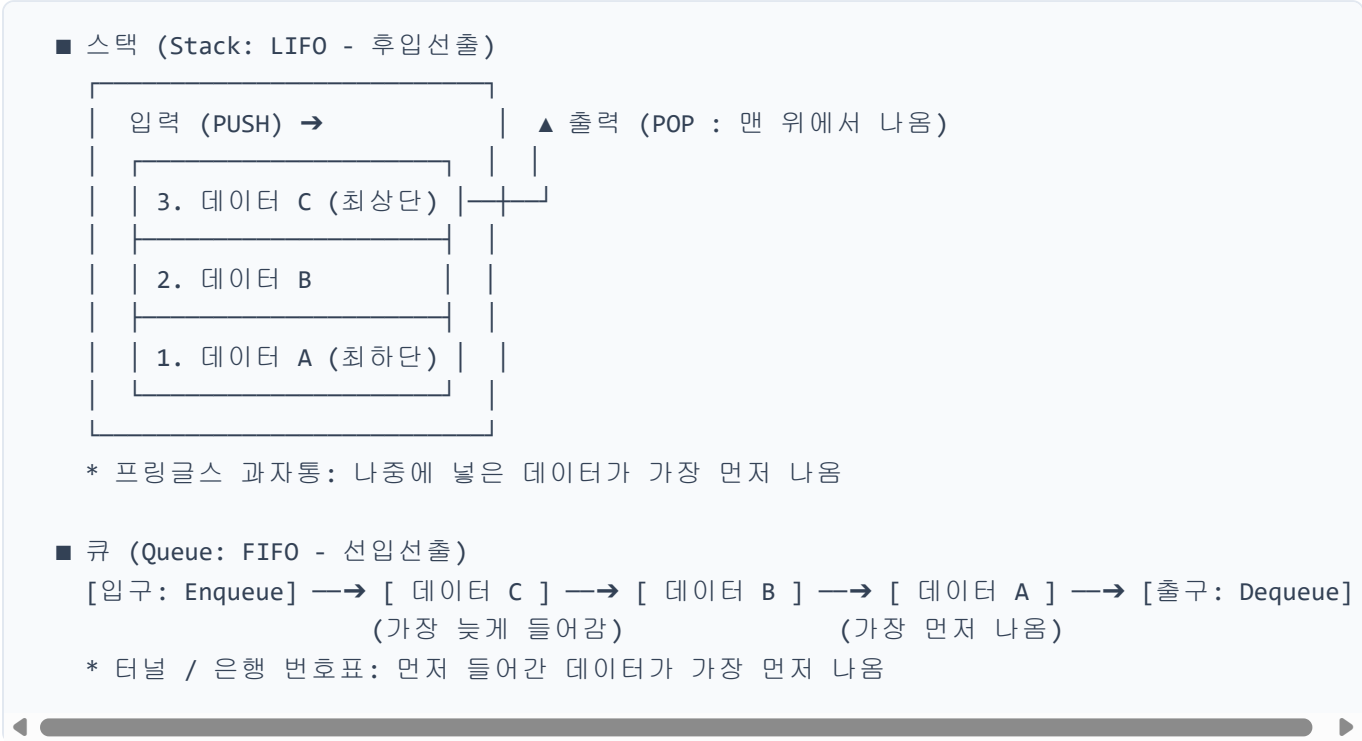
입력 (x)	출력 (y)	규칙 분석 (힌트 찾기)
2	**7**	2에 3배 하면 6 ➔ 1이 부족함 (+1)
3	**10**	3에 3배 하면 9 ➔ 1이 부족함 (+1)
5	**16**	5에 3배 하면 15 ➔ 1이 부족함 (+1)
8	**?**	$y = 3x + 1$ 규칙 성립! ➔ $3 \times 8 + 1 =$ **25**

- **규칙 도출 2단계 공식**:
  1. **기울기(변화율) 찾기**: x가 1 커질 때 y가 3씩 커지므로, 입력값에 **3을 곱하는 항($3x$)**이 기본입니다.
  2. **상수 보정하기**: $3 \times 2 = 6$인데 출력이 7이므로, 뒤에 $+1$을 더해 $f(x) = 3x + 1$을 완성합니다.

---

4. 핵심 자료구조: 스택과 큐 (Stack & Queue)

컴퓨터가 데이터를 어떤 순서로 쌓아두고 꺼낼 것인가를 결정하는 2대 기본 자료구조입니다.



4.1 스택 (Stack: LIFO - Last In, First Out)

- **특징**: 바닥이 막힌 상자 구조로, **가장 나중에 들어간 데이터가 가장 먼저 나옵니다**.
- **핵심 연산**: 데이터를 넣는 `Push`, 가장 위 데이터를 꺼내는 `Pop`.
- **실생활 및 알고리즘 활용**:
  - 웹 브라우저의 '뒤로 가기' 버튼
  - 문서 편집기의 '실행 취소(Ctrl+Z)'
  - ★ **수식의 괄호 짝 검사** `(()())`: 여는 괄호 `(`는 Push하고, 닫는 괄호 `)`가 나오면 Pop하여 짝이 맞는지 검사합니다.

4.2 큐 (Queue: FIFO - First In, First Out)

- **특징**: 양쪽이 뚫린 파이프 구조로, **가장 먼저 들어간 데이터가 가장 먼저 나옵니다**.
- **핵심 연산**: 줄 맨 뒤에 데이터를 넣는 `Enqueue`, 맨 앞 데이터를 내보내는 `Dequeue`.
- **실생활 활용**: 은행 번호표 대기 순서, 프린터 인쇄 대기열, 너비 우선 탐색(BFS).

---

5. 재귀 함수와 콜스택 (Recursion & Call Stack)

5.1 재귀 함수(Recursive Function)의 개념

- 함수가 종료되지 않고, 더 작은 크기의 문제를 해결하기 위해 **자기 자신을 다시 호출**하는 기법입니다.
- **★ 필수 조건 (기저 조건, Base Case)**:
  재귀 호출을 멈추고 값을 반환하는 종료 조건이 없으면 컴퓨터 메모리가 터지는 **스택 오버플로우(Stack Overflow)**가 발생합니다.

5.2 팩토리얼($N!$) 재귀 호출 추적

```
FUNCTION Factorial(N) {
    IF (N <= 1) {
        RETURN 1                    // 기저 조건 (종료 조건)
    } ELSE {
        RETURN N * Factorial(N - 1)  // 자기 자신을 N-1로 다시 호출
    }
}
```

- **호출 및 반환 흐름 (`Factorial(3)` 실행 시)**:
  1. `Factorial(3)` 호출 ➔ $3 \times$ Factorial(2) 대기
  2. `Factorial(2)` 호출 ➔ $2 \times$ Factorial(1) 대기
  3. `Factorial(1)` 호출 ➔ 기저 조건 만족! `1` 반환!
  4. 대기하던 값들이 역순으로 계산됨: $2 \times 1 = 2$ ➔ $3 \times 2 = \mathbf{6}$!

---

6. 알고리즘 효율성과 시간 복잡도 (Big-O Notation)

컴퓨터의 사양이 달라도 알고리즘 자체의 빠르기를 객관적으로 비교하기 위해, **입력 데이터의 개수 N이 증가할 때 연산 횟수가 증가하는 비율**을 수학적으로 표기한 것이 **빅오 표기법(Big-O)**입니다.

6.1 빅오 서열 비교 (왼쪽이 압도적으로 빠름)

$$O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(2^N)$$

표기법	명칭	일상 비유 및 코드 패턴	데이터 100만 개일 때 계산 횟수
$O(1)$	상수 시간	사물함 번호 알고 바로 열기 (배열 인덱스 조회)	단 **1번**

표기법	명칭	일상 비유 및 코드 패턴	데이터 100만 개일 때 계산 횟수
$O(\log N)$	로그 시간	스무고개 놀이, **이진 탐색 (Binary Search)**	단 **20번**!
$O(N)$	선형 시간	1부터 N까지 차례로 훑기 (단일 1중 루프)	**100만 번**
$O(N^2)$	제곱 시간	전교생끼리 서로 다 악수하기 (중첩 2중 루프, 버블 정렬)	**1조 번** (컴퓨터 다운!)

6.2 이진 탐색 (Binary Search) 의 핵심 원리

- **전제 조건**: **반드시 데이터가 사전에 오름차순으로 정렬**되어 있어야만 사용 가능!
- **동작 원리**: 중앙값을 확인하여 찾는 값이 왼쪽 또는 오른쪽에 있는지 판단 후, 매 단계마다 탐색 범위를 **절반($1/2$)씩 날려버립니다**.
- **★ 최대 비교 횟수 공식**: 데이터가 N개일 때, 최대 비교 횟수는 $\log_2 N$**을 올림한 정수값**입니다.
  - $N = 64$개 ➔ $2^6 = 64$ 이므로 최대 **6번** 만에 탐색 완료!
  - $N = 100$만 개 ➔ $2^{20} \approx 104$만이므로 단 **20번** 만에 탐색 완료!

---

7. 2D 바둑판 격자 로봇 시뮬레이션 (Grid Robot Simulation)

컴퓨팅 사고력 시험에서는 2차원 평면 좌표 위에서 로봇이나 캐릭터가 정해진 명령(앞으로 이동, 좌회전, 우회전 등)에 따라 이동할 때 최종 좌표나 경로를 묻는 문제가 빈출됩니다.

7.1 방향 벡터와 좌표 이동 규칙

- **기본 좌표 체계**:
  - 북쪽(위쪽)으로 1칸 이동: Y 좌표 $+1$ (남쪽으로 이동: Y 좌표 -1)
  - 동쪽(오른쪽)으로 1칸 이동: X 좌표 $+1$ (서쪽으로 이동: X 좌표 -1)
- **90도 회전 시 머리 방향의 변화**:
  - **우회전(R, 시계 방향)**: 북 ➔ 동 ➔ 남 ➔ 서 ➔ 북
  - **좌회전(L, 반시계 방향)**: 북 ➔ 서 ➔ 남 ➔ 동 ➔ 북

7.2 시험장 100% 득점 팁 (십자 방향키 그리기)

1. 머리로 회전 방향을 계산하면 좌회전과 우회전을 반대로 착각하기 쉽습니다.
2. 문제지 여백에 즉시 `동 서 남 북` **십자 방향키**를 작게 그려두세요.
3. 로봇이 회전할 때마다 **현재 바라보는 방향 화살표에 동그라미**를 치고, 전진할 때마다 해당 축의 숫자를 가감하면 절대 틀리지 않습니다.

✍️ 단원 마무리 100점 만점 실전 연습문제

[문제 1: 2중 중첩 반복문 상태 추적]

다음 의사코드를 실행한 후 최종적으로 화면에 출력되는 `TOTAL`의 값은 얼마입니까?

```
TOTAL = 0
FOR (A = 1 to 4) {
    FOR (B = 1 to A) {
        IF (B == 2) {
            TOTAL = TOTAL + 5
        } ELSE {
            TOTAL = TOTAL + 1
        }
    }
}
PRINT(TOTAL)
```

① 10
② 18
③ 22
④ 25

[문제 2: 스택(Stack) 자료구조 연산]

비어 있는 스택 *S*에 다음과 같은 연산을 순서대로 수행했습니다.

```
PUSH(10) → PUSH(20) → POP() → PUSH(30) → PUSH(40) → POP() → PUSH(50)
```

모든 연산이 끝난 후, 현재 스택 *S* 안에 남아 있는 **데이터들의 총합**은 얼마일까요?

① 60
② 80
③ 90
④ 110

[문제 3: 이진 탐색 비교 횟수 계산]

1부터 128까지 크기 순서대로 정렬된 128개의 정수 데이터가 들어있는 배열이 있습니다. 이 배열에서 특정한 목표값(Key)을 찾기 위해 **이진 탐색(Binary Search)**을 수행할 때, 목표값을 찾거나 데이터가 없음을 확인할 때까지 거쳐야 하는 **최대 비교 횟수**는 몇 회입니까?

① 7회
② 8회
③ 14회
④ 64회

[문제 4: 격자 로봇 방향 및 이동 추적]

로봇이 2차원 평면의 원점 `(0, 0)`에서 **북쪽(위쪽)**을 바라보고 서 있습니다. 다음 명령어 시퀀스를 순서대로 모두 실행했을 때, 로봇의 **최종 위치 좌표** `(X, Y)`는?
(단, `F`는 현재 바라보는 방향으로 1칸 전진, `R`은 오른쪽으로 90도 회전, `L`은 왼쪽으로 90도 회전입니다.)

> 명령어: `F ➔ F ➔ R ➔ F ➔ F ➔ F ➔ R ➔ F ➔ L ➔ F`

① (3, 1)
② (4, 1)
③ (3, 2)
④ (4, 2)

🎯 정답 및 단계별 풀이 해설

[문제 1 정답] ③

- **풀이 해설 (Trace Table 작성)**:
  - 외부 루프 A는 1부터 4까지, 내부 루프 B는 1부터 A까지 돕니다.
  - `A=1`일 때:
    - B=1: (B!=2) ➔ TOTAL = 0 + 1 = **1**
  - `A=2`일 때:
    - B=1: TOTAL = 1 + 1 = 2
    - B=2: (B==2) ➔ TOTAL = 2 + 5 = **7**
  - `A=3`일 때:
    - B=1: TOTAL = 7 + 1 = 8
    - B=2: (B==2) ➔ TOTAL = 8 + 5 = 13
    - B=3: TOTAL = 13 + 1 = **14**
  - `A=4`일 때:
    - B=1: TOTAL = 14 + 1 = 15
    - B=2: (B==2) ➔ TOTAL = 15 + 5 = 20
    - B=3: TOTAL = 20 + 1 = 21
    - B=4: TOTAL = 21 + 1 = **22**

- 루프 종료 후 출력값은 `22` 입니다.

[문제 2 정답] ③

- **풀이 해설 (스택 상태 추적)**:
  1. `PUSH(10)` ➔ 스택: `[10]`
  2. `PUSH(20)` ➔ 스택: `[10, 20]`
  3. `POP()` ➔ 가장 늦게 들어간 20이 빠져나옴 ➔ 스택: `[10]`
  4. `PUSH(30)` ➔ 스택: `[10, 30]`
  5. `PUSH(40)` ➔ 스택: `[10, 30, 40]`
  6. `POP()` ➔ 가장 늦게 들어간 40이 빠져나옴 ➔ 스택: `[10, 30]`
  7. `PUSH(50)` ➔ 스택: `[10, 30, 50]`
  - 최종 스택에 남아 있는 데이터는 **10, 30, 50**입니다.
  - 총합: $10 + 30 + 50 = \mathbf{90}$.

[문제 3 정답] ①

- **풀이 해설**:
  - 이진 탐색은 매 비교마다 검색 대상 데이터의 범위를 절반(1/2)으로 줄입니다.
  - 전체 데이터 개수가 $N = 128$일 때:
    - 1회 비교 후: 64개 남음
    - 2회 비교 후: 32개 남음
    - 3회 비교 후: 16개 남음
    - 4회 비교 후: 8개 남음
    - 5회 비교 후: 4개 남음
    - 6회 비교 후: 2개 남음
    - 7회 비교 후: 1개 남음 (탐색 종료)
  - 공식: $2^7 = 128$ 이므로, 최악의 경우 거쳐야 하는 최대 비교 횟수는 **7회**입니다!

[문제 4 정답] ②

- **풀이 해설 (방향 및 좌표 단계별 추적)**:
  - 시작: (0, 0), 방향: **북**
  - `F` ➔ (0, 1), 북
  - `F` ➔ (0, 2), 북
  - `R` ➔ 방향 우회전: **동** (좌표 변동 없음: (0, 2))
  - `F` ➔ (1, 2), 동
  - `F` ➔ (2, 2), 동
  - `F` ➔ (3, 2), 동
  - `R` ➔ 방향 우회전: **남** (좌표 변동 없음: (3, 2))
  - `F` ➔ (3, 1), 남 (남쪽이므로 Y좌표 1 감소)
  - `L` ➔ 남쪽에서 좌회전: **동** (좌표 변동 없음: (3, 1))
  - `F` ➔ (4, 1), 동 (동쪽이므로 X좌표 1 증가)

- 따라서 로봇의 최종 위치는 `(4, 1)`입니다!